MongoDB

Modificando documentos

Ya hemos visto que podemos añadir datos mediante la operación `insertOne`. Además, si queremos insertar más de un documento a la vez, podemos hacer uso de la operación `insertMany`, la cual admite un array de documentos.

Preparando una persona

Para este apartado, vamos a insertar dos veces la misma persona sobre la cual realizaremos las modificaciones:

```
use drfleming;
db.personas.insertMany([{nombre: "Manuel Ruiz", edad: 47, profesion: "Profesor"},
                        {nombre: "Manuel Ruiz", edad: 47, profesion: "Profesor"},
                        {nombre: "Andreu Ruiz", edad: 14, profesion: "Estudiante"}])
// {
//     acknowledged: true,
//     insertedIds: {
//         '0': ObjectId('6808cf1d0c0cbe759bb5f899'),
//         '1': ObjectId('6808cf1d0c0cbe759bb5f89a'),
//         '2': ObjectId('6808cf1d0c0cbe759bb5f89b')
//     }
// }
```

Para actualizar (y fusionar datos), se utilizan los métodos `updateOne` / `updateMany` dependiendo de cuantos documentos queremos que modifique. Ambos métodos requieren 2 parámetros: el primero es la consulta para averiguar sobre qué documentos, y en el segundo parámetro, los campos a modificar utilizando los operadores de actualización (en este ejemplo comenzaremos usando `$set`):

```
db.personas.updateOne(
    {nombre:"Manuel Ruiz"},
    {$set:{nombre:"Marina Ruiz", profesion:"Estudiante", salario: 123456}})
```

Al realizar la modificación, el *shell* nos devolverá información sobre cuantos documentos ha encontrado, modificado y más información:

```
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
```

Como hay más de una persona con el mismo nombre, al haber utilizado `updateOne` sólo modificará el primer documento que ha encontrado.

¡Cuidado!

En versiones antiguas de *MongoDB*, además de utilizar los operadores de actualización, podíamos pasarle como parámetro un documento, de manera que *MongoDB* realizaba un reemplazo de los campos, es decir, si en el origen había 100 campos y en la operación de modificación sólo poníamos 2, el resultado únicamente contendría 2 campos. Es por ello, que ahora es obligatorio utilizar los operadores.

Si cuando vamos a actualizar, en el criterio de selección no encuentra el documento sobre el que hacer los cambios, no se realiza ninguna acción.

Si quisiéramos que en el caso de no encontrar nada insertase un nuevo documento, acción conocida como ***upsert*** (*update + insert*), hay que pasarle un tercer parámetro al método con el objeto `{upsert:true}`. Si encuentra el documento, lo modificará, pero si no, creará uno nuevo:

```
db.personas.updateOne({nombre:"Andreu Ruiz"},
                      {$set:{name:"Andreu Ruiz", bsky: "@andreuRuiz"}},
                      {upsert: true})
```

Operadores de actualización

MongoDB ofrece un conjunto de operadores para simplificar la modificación de campos.

El operador más utilizado es el operador `$set`, el cual admite los campos que se van a modificar. Si el campo no existe, lo creará.

Por ejemplo, para modificar el `salario` haríamos:

```
db.personas.updateOne({nombre:"Manuel Ruiz"}, {$set:{salario: 1000000}})
```

Mediante `$inc` podemos incrementar (o decrementar en negativo) el valor de una variable:

```
db.personas.updateOne({nombre:"Manuel Ruiz"}, {$inc:{salario: 1000}})
db.personas.updateOne({nombre:"Marina Ruiz"}, {$inc:{salario: -500}})
```

Para eliminar un campo de un documento, usaremos el operador `$unset`.

De este modo, para eliminar el campo *bsky* de una persona haríamos:

```
db.personas.updateOne({nombre:"Manuel Ruiz"}, {$unset:{bsky:''}})
```

Otros operadores que podemos utilizar son `$mul`, `$min`, `$max` y `$currentDate`. Podemos consultar todos los operadores disponibles en https://www.mongodb.com/docs/manual/reference/operator/update/

Autoevaluación

Tras realizar la siguiente operación sobre una colección vacía:

```
db.personas.updateOne(
    {nombre:'yo'},
    {'$set':{'hobbies':['gaming', 'sofing']}},
    {upsert: true} );
```

¿Cuál será el estado de la colección?

Finalmente, un caso particular de las actualizaciones es la posibilidad de renombrar un campo mediante el operador `$rename`:

```
db.personas.updateMany( {_id:1},
    {$rename:{'nickname':'alias', 'cell':'movil'}})
```

Podemos consultar todas las opciones de configuración de una actualización en https://www.mongodb.com/docs/manual/reference/method/db.collection.update/.

Reemplazando un documento

Desde Mongo 4.2, podemos emplear la operación `replaceOne` para sustituir todo el contenido por el documento que le pasamos como parámetro.

Así pues, si queremos sustituir el documento de `Andreu Ruiz`, borrando los campos que tuviese y remplazándolos por nuevos podríamos hacer:

```
db.personas.replaceOne({nombre:"Andreu Ruiz"},
    {nombre:"Andreu", apellidos:"Ruiz",
     hobbies:["baloncesto","videojuegos","spiderman"]})
```

Control de la concurrencia

Cuando se hace una actualización múltiple, *MongoDB* no realiza la operación de manera atómica (a no ser que utilicemos transacciones desde el *driver*), lo que provoca que se puedan producir pausas (*pause yielding*). Cada documento en sí es atómico, por lo que ninguno se va a quedar a la mitad.

MongoDB ofrece el método `findAndModify` para **encontrar y modificar un documento de manera atómica**, y así evitar que, entre la búsqueda y la modificación, el estado del documento se vea afectado. Además, devuelve el documento modificado. Un caso de uso muy común es para contadores y casos similares.

```
db.personas.findAndModify({
  query: {nombre:"Marina Ruiz"},
  update: {$inc:{salario:100, edad:-30}},
  new: true
})
// {
//   _id: ObjectId('6808cf1d0c0cbe759bb5f899'),
//   nombre: 'Marina Ruiz',
//   edad: 17,
//   profesion: 'Estudiante',
//   salario: 123556
// }
```

Por defecto, el documento devuelto será el resultado que ha encontrado con la consulta. Si queremos que nos devuelva el documento modificado con los cambios deseados, necesitamos utilizar el parámetro `new` a `true`. Si no lo indicamos o lo ponemos a `false`, tendremos el comportamiento por defecto, es decir, el documento original.

Borrando documentos

Para borrar, usaremos los métodos `deleteOne` o `deleteMany`, los cuales funcionan de manera similar a `findOne` y `find`. Si no pasamos ningún parámetro, `deleteOne` borrará el primer documento, o en el caso de `deleteMany` toda la colección documento a documento. Si le pasamos un parámetro, éste será el criterio de selección de documentos a eliminar.

```
db.personas.deleteOne({nombre:"Marina Ruiz"})
db.personas.deleteMany({nombre:"Marina Ruiz"})
```

Al eliminar un documento, no podemos olvidar que cualquier referencia al documento que exista en la base de datos seguirá existiendo. Por este motivo, manualmente también hay que eliminar o modificar esas referencias.

Eliminar un campo

Recordad que eliminar un determinado campo de un documento no se considera una operación de borrado, sino una actualización mediante el operador `$unset`.

Si queremos borrar toda la colección, es más eficiente usar el método `drop`, ya que también elimina los índices.

```
db.personas.drop()
```

Finalmente, para eliminar una base de datos usaremos el comando `db.dropDatabase()`, que eliminará la base de datos sobre la que nos encontremos (en nuestro caso, en `drfleming`):

```
db.dropDatabase()
```

Trabajando con *Arrays*

Para trabajar con *arrays* necesitamos nuevos operadores que nos permitan tanto introducir como eliminar elementos de una manera más sencilla que sustituir todos los elementos del array.

Los operadores que podemos emplear para trabajar con *arrays* son:

Operador	Propósito
$push	Añade uno o varios elementos
$addToSet	Añade un elemento sin duplicados
$pull	Elimina un elemento
$pullAll	Elimina varios elementos
$pop	Elimina el primer o el último

Preparando los ejemplos

Para trabajar con los *arrays*, vamos a suponer que tenemos una colección de enlaces donde vamos a almacenar un documento por cada site, con un atributo `tags` con etiquetas sobre el enlace en cuestión

```
db.enlaces.insertOne({titulo:"www.google.es", tags:["mapas", "videos"]})
```

De modo que tendríamos el siguiente documento:

```
{ _id: ObjectId("633c60e8ac452ac9d7f9fe74"),
  titulo: 'www.google.es',
  tags: [ 'mapas', 'videos' ] }
```

Añadiendo elementos

Si queremos añadir uno o varios elementos, usaremos el operador $push. Cuando queremos añadir varios elementos a la vez, mediante el operador $each le pasamos un array con los datos:

```
db.enlaces.updateMany({titulo:"www.google.es"},
                      {$push: {tags:"blog"}} )
db.enlaces.updateMany({titulo:"www.google.es"},
                      {$push: {tags:{$each:["calendario", "email",
"mapas"]}}})
```

Al hacer estar modificación, el resultado del documento sería:

```
{ _id: ObjectId("633c61b5ac452ac9d7f9fe75"),
  titulo: 'www.google.es',
  tags: [ 'mapas', 'videos', 'blog', 'calendario', 'email', 'mapas' ] }
```

Al utilizar `$push` no se tiene en cuenta lo que contiene el *array*, por tanto, si un elemento ya existe, se repetirá y tendremos duplicados. Si queremos evitar los duplicados, usaremos $addToSet:

```
db.enlaces.updateMany({titulo:"www.google.es"},
                  {$addToSet:{tags:"buscador"}})
```

Si queremos añadir más de un campo a la vez sin duplicados, debemos anidar el operador `$each` igual que hemos hecho antes:

```
db.enlaces.updateMany({titulo:"www.google.es"},
                  {$addToSet:{tags:{$each:["drive", "traductor"]}}})
```

En cambio, si queremos eliminar elementos de un array, usaremos el operador $pull:

```
db.enlaces.updateMany({titulo:"www.google.es"},
                  {$pull:{tags:"traductor"}})
```

Similar al caso anterior, con $pullAll, eliminaremos varios elementos de una sola vez:

```
db.enlaces.updateMany({titulo:"www.google.es"},
                  {$pullAll:{tags:["calendario", "email"]}})
```

Otra manera de eliminar elementos del array es mediante $pop, el cual elimina el primero (-1) o el último (1) elemento del array:

```
db.enlaces.updateMany({titulo:"www.google.es"}, {$pop:{tags:-1}})
```

Operador posicional

Por último, tenemos el operador posicional, el cual se expresa con el símbolo $ y nos permite modificar el elemento que ocupa una determinada posición del array sin saber exactamente cuál es esa posición.

Supongamos que tenemos las calificaciones de los estudiantes (colección `students`) en un documento con una estructura similar a la siguiente:

```
{ "_id" : 1, "notas" : [ 80, 85, 90 ] }
```

y queremos cambiar la calificación de 80 por 82. Mediante el operador posicional haremos:

```
db.students.updateOne( { _id: 1, notas: 80 }, { $set: { "notas.$" : 82 } } )
```

De manera similar, si queremos modificar parte de un documento el cual forma parte de un array, debemos usar la notación punto tras el $.

Por ejemplo, supongamos que tenemos estas calificaciones de un determinado alumno, las cuales forman parte de un objeto dentro de un *array*:

```
{ "_id" : 4, "notas" :
  [ { nota: 80, media: 75 },
    { nota: 85, media: 90 },
    { nota: 90, media: 85 } ] }
```

Podemos observar cómo tenemos cada calificación como parte de un objeto dentro de un array. Si queremos cambiar el valor de `media` a 89 de la calificación cuya `nota` es 85, haremos:

```
db.students.updateOne( { _id: 4, "notas.nota": 85 }, { $set: {
"notas.$.media" : 89 } } )
```

Es decir, el $ referencia al documento que ha cumplido el filtro de búsqueda.

Más operadores posicionales

Además del operador posicional $, tenemos disponible el operador posicional `$[]` que indica que afecta a todos los elementos del array, y el operador posicional `$[identificador]` que identifica que elementos del array cumplen una condición para su filtrado.

Podemos consultar toda la documentación disponible sobre estos operadores en http://docs.mongodb.org/manual/reference/operator/update-array/

Administración

Empleando JavaScript

Ya hemos comentado que el *shell* utiliza *JavaScript* como lenguaje de interacción, por lo que podemos almacenar los comandos en un script externo y ejecutarlo mediante `load()`:

```
load("scripts/misDatos.js");
load("/data/db/scripts/misDatos.js");
```

MongoDB Compass

Si utilizamos *mongosh* desde *MongoDB Compass*, no podemos hacer uso del comando `load`.
Sí que podemos crear variables y fragmentos de código en *Javascript*, sabiendo que para añadir un salto de línea emplearemos `Shift+Enter`.

Si hacemos una referencia relativa, lo hace respecto a la ruta desde la cual se ejecuta el *shell* `mongosh`.

Otra manera de lanzar un *script* es hacerlo desde la línea de comandos, pasándole como segundo parámetro el script a ejecutar:

```
mongosh basededatos misDatos.js
```

Si el código a ejecutar no necesita almacenarse en un *script* externo, el propio *shell* permite introducir instrucciones en varias líneas:

```
for (var i=0;i<10;i++) {
  db.espias.insertOne({"nombre":"James Bond " + i, "agente":"00" + i});
}
// {
//   acknowledged: true,
//   insertedId: ObjectId('67fbf408b496860bf5c367a9')
// }
db.espias.find()
// [
//   {
//     _id: ObjectId('67fbf408b496860bf5c367a0'),
//     nombre: 'James Bond 0',
//     agente: '000'
//   }
//   {
//     _id: ObjectId('67fbf408b496860bf5c367a1'),
//     nombre: 'James Bond 1',
//     agente: '001'
//   }
//   ...
//   {
//     _id: ObjectId('67fbf408b496860bf5c367a9'),
//     nombre: 'James Bond 9',
//     agente: '009'
//   }
// ]
```

Importando y exportando

Además del propio servidor de *MongoDB* y el cliente para conectarse a él, *MongoDB* ofrece un conjunto de herramientas para interactuar con las bases de datos, permitiendo crear y restaurar copias de seguridad.

Si estamos interesados en introducir o exportar una colección de datos mediante JSON, podemos emplear los comandos `mongoimport` y `mongoexport`:

```
mongoimport

--uri "mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/
nombreBaseDatos "

--collection coleccion --file nombreFichero.json
```

```
mongoexport

--uri "mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/
nombreBaseDatos "

--collection  coleccion --out nombreFichero.json
```

Estas herramientas interactúan con datos JSON y no sobre toda la base de datos.

Un caso particular y muy común es importar datos que se encuentran en formato CSV/TSV. Para ello, emplearemos el parámetro `--type csv` o `--type tsv` según el separador de los campos sea la coma o el tabulador:

```
mongoimport

--uri
"mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/nombreBaseDatos"

 --type tsv -d test --collection micoleccion --headerline

 --drop documento.tsv
```

En vez de realizar un *export*, es más conveniente realizar un *backup* en binario mediante `mongodump`, el cual genera ficheros BSON. Estos archivos posteriormente se restauran mediante `mongorestore`:

```
mongodump

--uri "mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/
nombreBaseDatos"

 --collection micoleccion --out  nombreFichero.bson


mongorestore

--uri "mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/
nombreBaseDatos" nombreFichero.bson
```

Autoevaluación

Intenta exportar los datos de la base de datos `sample_training` desde *MongoAtlas*. Verás que ha creado una carpeta que contiene dos archivos ¿Cuáles son? ¿Qué contiene cada uno de ellos y cuál es su formato?

Si necesitamos transformar un fichero BSON a JSON (de binario a texto), tenemos el comando `bsondump`:

```
bsondump file.bson > file.json
```

Info

Más información sobre copias de seguridad en https://www.mongodb.com/docs/manual/core/backups/.

Para poder trabajar con *MongoDB* desde cualquier aplicación necesitamos un *driver*. *MongoDB* ofrece drivers oficiales para casi todos los lenguajes de programación actuales.

Monitorización

Tanto `mongostat` como `mongotop` permiten visualizar el estado del servidor *MongoDB*, así como algunas estadísticas sobre su rendimiento. Si trabajamos con *MongoAtlas* estas herramientas están integradas en las diferentes herramientas de monitorización de la plataforma.

Modelado documental

Una de las características más importantes al utilizar un sistema documental es que mediante un documento podemos mantener toda la información que se utiliza junta en un único documento, sin necesidad de separar los datos en diferentes colecciones.

Dependiendo del tipo de relación entre dos documentos, normalizaremos los datos para minimizar la redundancia, pero manteniendo en la medida de lo posible que mediante operaciones atómicas se mantenga la integridad de los datos. Para ello, bien crearemos referencias entre dos documentos o embeberemos un documento dentro de otro, pero intentando que la información más utilizada quepa en un único documento.

MongoDB es una base de datos documental, no relacional, donde el esquema no se debe basar en el uso de claves ajenas/*joins*, ya que realmente no existen. A la hora de diseñar un esquema, si nos encontramos que el esquema está en 3FN o si cuando hacemos consultas estamos teniendo que realizar varias consultas de manera programática (primero acceder a una tabla, con ese `_id` ir a otra tabla, etc… o hacemos un uso extensivo del operador `$lookup` mediante el *framework* de agregación) es que no estamos siguiendo el enfoque adecuado.

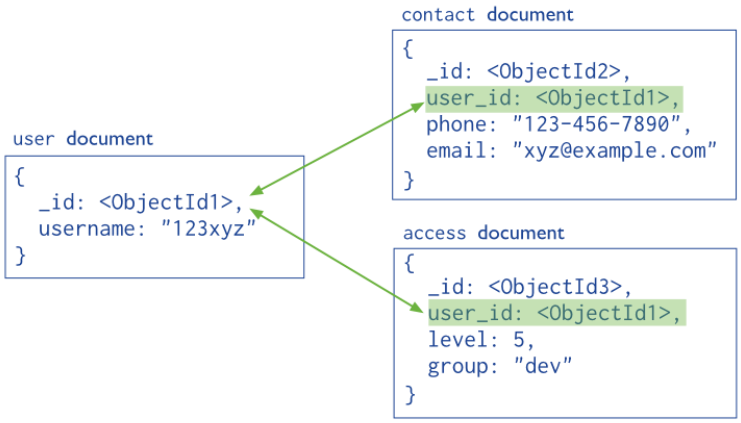
En resumen, diseñar un buen modelo de datos documental supondrá que nuestro código será más legible y mantenible, así como tener un mayor rendimiento de nuestras aplicaciones, aunque conlleve la redundancia de datos.

Las aplicaciones que emplean *MongoDB* utilizan dos técnicas para relacionar documentos:

- Crear referencias
- Embeber documentos

Referencias manuales

De manera similar a una base de datos relacional, se almacena el campo `_id` de un documento en otro documento a modo de clave ajena. De este modo, la aplicación realiza una segunda consulta para obtener los datos relacionados. Estas referencias son sencillas y suficientes para la mayoría de los casos de uso.



Referencias manuales

Por ejemplo, si nos basamos en el gráfico anterior, podemos conseguir referenciar manualmente estos objetos del siguiente modo:

```
var idUsuario = ObjectId();

db.usuario.insertOne({
  _id: idUsuario,
  nombre: "123xyz"
});

db.contacto.insertOne({
  usuario_id: idUsuario,
```

```
  telefono: "123 456 7890",
  email: "xyz@ejemplo.com"
});
```

Para relacionar los dos documentos, haremos uso de la operación `$lookup` para hacer el *join*, o haremos una segunda consulta para la segunda colección. Un ejemplo de *join* mediante `$lookup`:

```
db.usuario.aggregate([ {
  $lookup: {
    from: "contacto",
    localField: "_id",
    foreignField: "usuario_id",
    as: "contacto_data"
  }
} ])
```

Y como resultado obtenemos un documento con el usuario y la información del contacto dentro de un array embebido (aunque en este ejemplo sólo tenemos un contacto para el usuario)

```
{ _id: ObjectId("634589696e96ece54fbcbca2"),
  nombre: '123xyz',
  contacto_data:
   [ { _id: ObjectId("634589696e96ece54fbcbca3"),
       usuario_id: ObjectId("634589696e96ece54fbcbca2"),
       telefono: '123 456 7890',
       email: 'xyz@ejemplo.com' } ] }
```

El operador `$lookup` lo estudiaremos en profundidad en el apartado de *Consultas agregadas*.

Datos embebidos

En cambio, si dentro de un documento almacenamos los datos mediante subdocumentos, ya sea dentro de un atributo o un array, podremos obtener todos los datos mediante un único acceso, sin necesidad de claves ajenas ni comprobaciones de integridad referencial.

Datos embebidos

Generalmente, emplearemos datos embebidos cuando tengamos:

- relaciones "contiene" entre entidades, entre relaciones de documentos "uno a uno" o "uno a pocos".
- relaciones "uno a muchos" entre entidades. En estas relaciones los documentos hijo (o "muchos") siempre aparecen dentro del contexto del padre o del documento "uno".

Los datos embebidos/incrustados ofrecen mejor rendimiento al permitir obtener los datos mediante una única operación, así como modificar datos relacionados en una sola operación atómica de escritura (sin necesidad de transacciones)

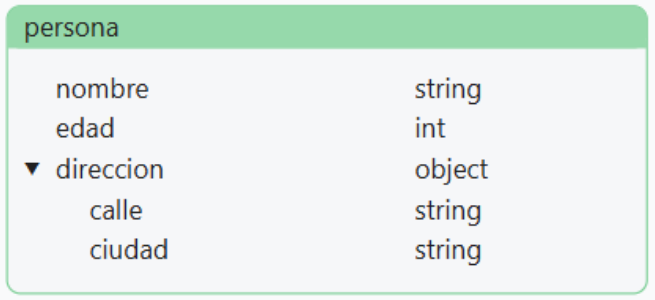
Un aspecto a tener en cuenta es que un documento BSON puede contener un máximo de 16MB. Si quisiéramos que un atributo contenga más información, tendríamos que utilizar el API de GridFS.

Vamos a estudiar en detalle cada uno de los tipos de relaciones, para intentar clarificar cuando es conveniente utilizar referencias o datos embebidos.

Relaciones 1:1

Cuando existe una relación 1:1, como pueda ser entre `Persona` y `Curriculum`, o `Persona` y `Direccion` hay que embeber un documento dentro del otro, como parte de un atributo.

- **Estructura**



Relación 1:1 persona-dirección

- **Documento de ejemplo**

persona.json
```
{
  nombre: "Manuel",
  edad: 18,
  direccion: {
    calle: "Secreta",
    ciudad: "Oviedo"
  }
}
```

La principal ventaja de este planteamiento es que mediante una única consulta podemos obtener tanto los detalles del usuario como su dirección.

Un par de aspectos que nos pueden llevar a no embeberlos son:

- la frecuencia de acceso. Si a uno de ellos se accede raramente, puede que convenga tenerlos separados para liberar memoria.
- el tamaño de los elementos. Si hay uno que es mucho más grande que el otro, o uno lo modificamos muchas más veces que el otro, para que cada vez que hagamos un cambio en un documento no tengamos que modificar el otro será mejor separarlos en documentos separados.

Pero siempre teniendo en cuenta la atomicidad de los datos, ya que, si necesitamos modificar los dos documentos al mismo tiempo, tendremos que embeber uno dentro del otro.

Relaciones 1:N

Vamos a distinguir tres tipos:

- **1 a pocos (1:F)** (*one to few*), como, por ejemplo, dentro de un blog, la relación entre `Mensaje` y `Comentario`. En este caso, la mejor solución es crear un array dentro de la entidad 1 (en nuestro caso, `Mensaje`). De este modo, el `Mensaje` contiene un array de `Comentario`:
- **Estructura**

Relación 1:F mensaje-comentarios

- **Documento de ejemplo**

mensaje.json
```
{
  "titulo": "La broma asesina",
  "url": "http://es.wikipedia.org/wiki/Batman:_The_Killing_Joke",
  "texto": "La dualidad de Batman y Joker",
  "comentarios": [
    {
      "autor": "Bruce Wayne",
      "fecha": ISODate("2022-10-11T09:31:32Z"),
      "comentario": "A mí me encantó"
    },
    {
      "autor": "Bruno Díaz",
      "fecha": ISODate("2022-10-11T10:07:28Z"),
      "comentario": "El mejor"
    }
  ]
}
```

Vigilar el tamaño

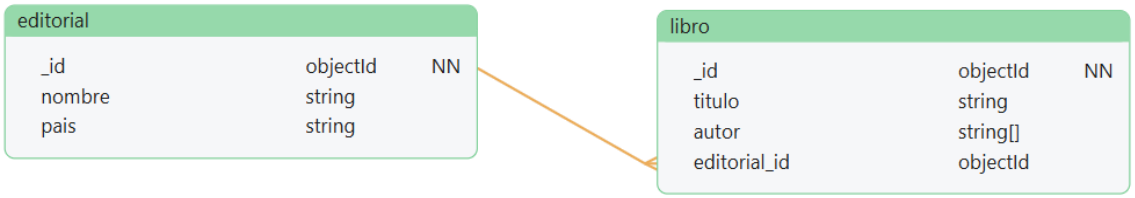
Hay que tener siempre en mente la restricción de los 16 MB de BSON. Si vamos a embeber muchos documentos y estos son grandes, hay que vigilar no llegar a dicho límite.

Si queremos saber cuánto ocupa un documento, en el *shell* podemos obtener su tamaño mediante el comando `bsonsize`:

```
> var p = db.people.findOne()
> bsonsize(p)
< 96
```

- **1 a muchos (1:N)** (*one to many*), como puede ser entre `Editorial` y `Libro`. Para este tipo de relación es mejor usar referencias entre los documentos colocando la referencia en el lado del muchos:

- **Estructura**



Relación 1:N editorial-libro

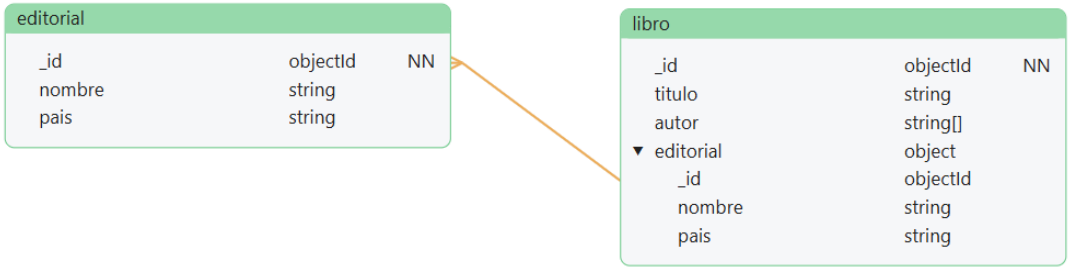
- **Documento de ejemplo**

editorial.json
```
{
  _id: 1,
  nombre: "O'Reilly",
  pais: "EE.UU."
}
```

libros.json
```
{
  _id: 1234,
  titulo: "MongoDB: The Definitive Guide",
  autor: [ "Kristina Chodorow", "Mike Dirolf" ],
  numPaginas: 216,
  editorial_id: 1,
},
{
  _id: 1235,
  titulo: "50 Tips and Tricks for MongoDB Developer",
  autor: "Kristina Chodorow",
  numPaginas: 68,
  editorial_id: 1,
}
```

Si cada vez que recuperamos un libro queremos tener el nombre de la editorial y con una sola consulta recuperar todos los datos, en vez poner la referencia a la editorial, podemos embeber toda la información (esto se conoce como el patrón *referencia extendida*), a costa de que un futuro cambio en el nombre de la editorial conlleve modificar muchos libros:

- **Estructura**



Relación 1:N editorial-libro extendido

- **Documento de ejemplo**

libros-extendido.json
```
{
  _id: 1234,
  titulo: "MongoDB: The Definitive Guide",
  autor: [ "Kristina Chodorow", "Mike Dirolf" ],
  numPaginas: 216,
  editorial: {
    _id: 1,
    nombre: "O'Reilly",
    pais: "EE.UU."
  }
},{
  _id: 1235,
  titulo: "50 Tips and Tricks for MongoDB Developer",
  autor: "Kristina Chodorow",
```

```
  numPaginas: 68,
  editorial: {
    _id: 1,
    nombre: "O'Reilly",
    pais: "EE.UU."
  }
}
```

Un caso particular en las relaciones uno a muchos que se traducen en documentos embebidos es cuando la información que nos interesa tiene un valor concreto en un momento determinado. Por ejemplo, dentro de un pedido, el precio de los productos debe embeberse, ya que, si en un futuro se modifica el precio de un producto determinado debido a una oferta, el pedido realizado no debe modificar su precio total.

Del mismo modo, al almacenar la dirección de una persona, también es conveniente embeberla. No queremos que la dirección de envío de un pedido ya enviado se modifique si un usuario cambia sus datos personales.

En cambio, si necesitamos acceder por separado a los diferentes objetos de una relación, puede que nos convenga separarlo en dos colecciones distintas, aunque luego tengamos que hacer un *join*.

- **1 a muchísimos/tropecientos (1:S)** (*one to squillions/zillions*), como puede ser entre una aplicación y los mensajes del log, los cuales pueden llegar a tener un volumen de millones de mensaje por aplicación. Teniendo siempre en mente la restricción de los 16MB de BSON, podemos modelar estas relaciones mediante un array de referencias:

aplicacion.json
```
{
  _id: ObjectId("111111"),
  nombre: "Gestión de clientes",
}
```

logs.json
```
{
  _id: ObjectId("123456"),
  app: ObjectId("111111"),
  actividad: "Alta Cliente",
  mensaje: "El cliente XXX se ha creado correctamente"
  date: ISODate("2022-10-12")
},{
  _id: ObjectId("123457"),
  app: ObjectId("111111"),
  actividad: "Modificación Cliente",
  mensaje: "No se ha podido modificar el XXX por un error del sistema"
  date: ISODate("2022-10-12")
},
```

De esta manera, pasamos la relación de 1 a muchos a realmente ser de muchos a 1, donde cada mensaje de log almacena la aplicación a la que pertenecen, y ya no tenemos que mantener un array de logs dentro de cada aplicación.

Relaciones N:M

Más que relaciones muchos a muchos, suelen ser relaciones pocos a pocos, como, por ejemplo, `Libro` y `Autor`, o `Profesor` y `Estudiante`.

Supongamos que tenemos libros modelados de la siguiente manera:

libro.json
```
{
  _id: 1,
```

```
  titulo: "La historia interminable",
  anyo: 1979
}
```

Y autores con la siguiente estructura:

autor.json
```
{
  _id: 1,
  nombre: "Michael Ende",
  pais: "Alemania"
}
```

Podemos resolver esta relación de tres maneras:

1. Siguiendo un enfoque relacional, empleando un documento como la entidad que agrupa con referencias manuales a los dos documentos.

libro-autor.json
```
{
  _id: 1,
  autor_id: 1,
  libro_id: 1
}
```

Este enfoque se desaconseja porque necesita acceder a tres colecciones para obtener toda la información.

2. Mediante 2 documentos, cada uno con un array que contenga los identificadores del otro documento (*2 Way Embedding*). Hay que tener cuidado porque podemos tener problemas de inconsistencia de datos si no actualizamos correctamente.

libro-con-autores.json
```
{
  _id: 1,
  titulo: "La historia interminable",
  anyo: 1979,
  autores: [1]
},{
  _id: 2,
  titulo: "Momo",
  anyo: 1973,
  autores: [1]
}
```

autor-con-libros.json
```
{
  _id: 1,
  nombre: "Michael Ende",
  pais: "Alemania",
  libros: [1, 2]
}
```

Aunque tenemos información redundante

Al utilizar referencias para representar una relación muchos a muchos, realmente sólo necesitamos un array de referencias en sólo una de las colecciones. Si nos fijamos en el atributo `libros` de la colección `autor-con-libros` ¿Podríamos obtener todos los libros de un autor? Por supuesto. Y sí quisiéramos obtener todos los libros de autores alemanes, aunque nos tocase realizar un *join*, también lo podríamos conseguir.

Dicho esto, aunque podemos tener un array en ambas colecciones, dependiendo del caso, puede conllevar una sobrecarga extra innecesaria.

3. Embeber un documento dentro de otro (*One Way Embedding*). Por ejemplo:

libro-con-autores-embebidos.json
```
{
    _id: 1,
    titulo: "La historia interminable",
    anyo: 1979,
    autores: [{nombre:"Michael Ende", pais:"Alemania"}]
},{
    _id: 2,
    titulo: "Momo",
    anyo: 1973,
    autores: [{nombre:"Michael Ende", pais:"Alemania"}]
}
```

En principio este enfoque no se recomienda porque el documento puede crecer mucho y provocar anomalías de modificaciones donde la información no es consistente. Si se opta por esta solución, hay que tener en cuenta que si un documento depende de otro para su creación. Por ejemplo, en una colección que relacione profesor con alumnos, si metemos los profesores dentro de los estudiantes, no vamos a poder dar de alta a profesores sin haber dado de alta previamente a un alumno.

A modo de resumen, en las relaciones N:M, hay que establecer el tamaño de N y M. Si N como máximo vale 3 y M 500000, entonces deberíamos seguir un enfoque de embeber la N dentro de la M (*One Way Embedding*).

En cambio, si N vale 3 y M vale 5, entonces podemos hacer que ambos embeban al otro documento (*Two Way Embedding*).

Rendimiento e Integridad

A modo de resumen, embeber documentos ofrece un mejor rendimiento que referenciar, ya que con una única operación (ya sea una lectura o una escritura) podemos acceder a varios documentos.

Cuidado con los *arrays*

Los *arrays* no pueden crecer de forma descontrolada. Si hay un par de cientos de documentos en el lado de N, no hay que embeberlos. Si hay más de unos pocos miles de documentos en el lado de N, no hay que usar un array de referencias. *Arrays* con una alta cardinalidad son una clara pista para no embeber.

Autoevaluación

A partir del siguiente supuesto, realiza el modelo documental necesario.

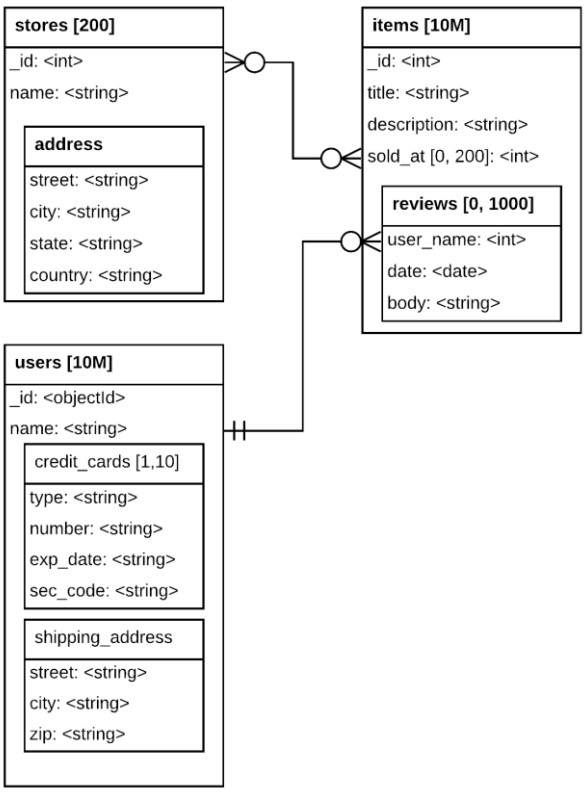
Enunciado

Tenemos **almacenes**, de los cuales, además de su nombre, necesitamos guardar su **dirección**, con su calle, ciudad, provincia y país. No tendremos más de 200 almacenes. Cada uno de los almacenes almacenará **productos**, hasta un máximo de 10 millones por almacén. De cada producto, además de su código, nombre y descripción, guardaremos los últimos 200 **precios** a los cuales se ha vendido cada producto. Además, cada producto puede tener **opiniones** creadas por usuarios, hasta un máximo de 1.000. Para estas opiniones, además de los datos del usuario, almacenaremos la fecha y el texto de la opinión.

Respecto a los **usuarios**, el sistema espera almacenar un máximo de 10 millones de usuarios, y de estos, además del nombre, guardaremos la **dirección** completa (calle, ciudad y código postal) donde enviar los productos, así como hasta un máximo de 10 tarjetas de crédito con la que realiza los pagos. De las **tarjetas**, almacenaremos su tipo, número bancario, fecha de expedición y código de seguridad.

¿Cuántas colecciones crearías?

Solución



Consultas agregadas

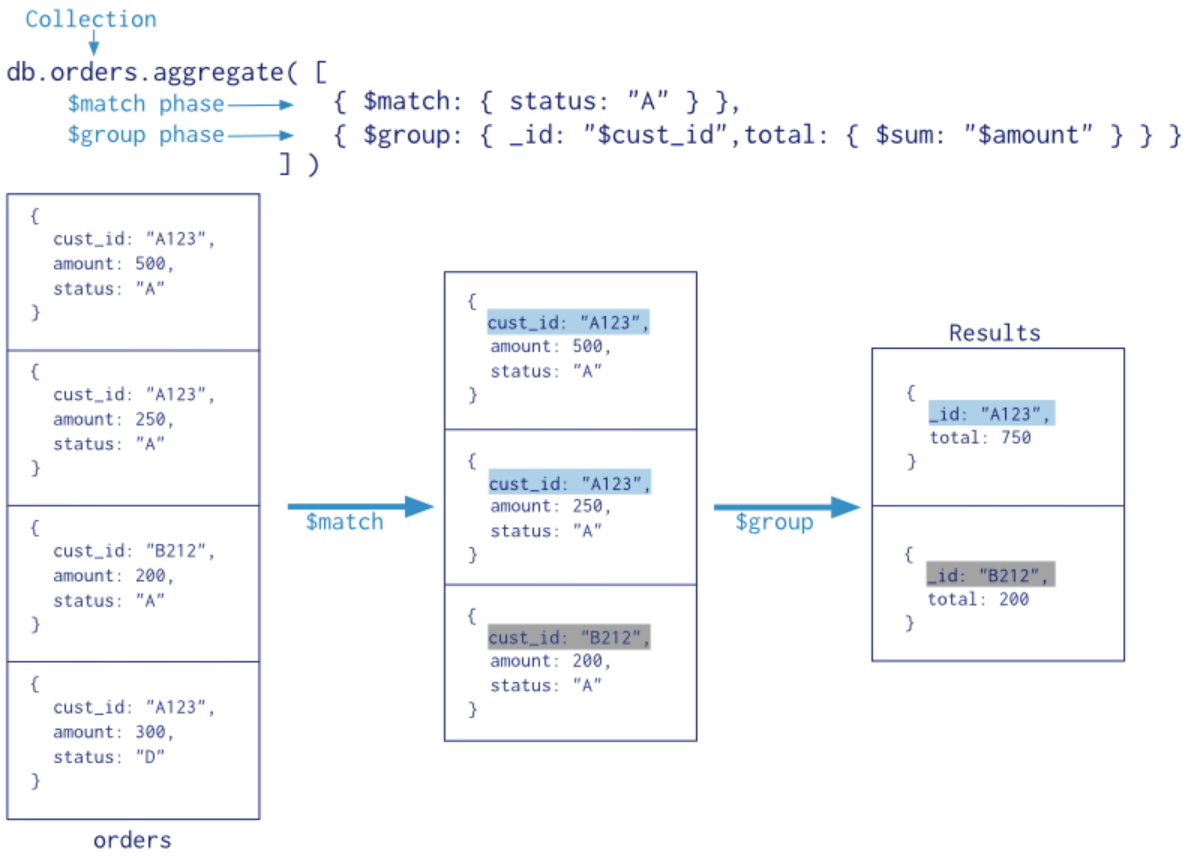
Las agregaciones usan un *pipeline*, conocido como ***Aggregation Pipeline***, de ahí que el método `aggregate` use un array con `[ ]` donde cada elemento es una fase del *pipeline*, de modo que la salida de una fase es la entrada de la siguiente:

```
db.coleccion.aggregate([fase1, fase2, ... faseN])
```

Cuidado con el tamaño

El resultado del *pipeline* es un documento y por lo tanto está sujeto a la restricción de BSON, que limita su tamaño a 16MB.

En la siguiente imagen se resumen los pasos de una agrupación donde primero se eligen los elementos que vamos a agrupar mediante `$match`, el resultado saliente se agrupan con `$group`, y sobre los agrupado mediante `$sum` se calcula el total:



Ejemplo de pipeline con $match y $group

Al realizar un *pipeline* dividimos las consultas en fases, donde cada fase utiliza un operador para realizar una transformación. Aunque no hay límite en el número de fases en una consulta, es importante destacar que **el orden importa**, y que hay optimizaciones para ayudar a que el *pipeline* tenga un mejor rendimiento (por ejemplo, hacer un `$match` al principio para reducir la cantidad de datos)

Operadores del pipeline

Antes de nada, cabe destacar que las fases se pueden repetir, por lo que una consulta puede repetir operadores para encadenar diferentes acciones.

A continuación, vamos a estudiar todos estos operadores:

Operador	Descripción	Cardinalidad
`$project`	Proyección de campos, es decir, propiedades en las que estamos interesados. También nos permite modificar un documento, o crear un subdocumento (*reshape*)	1:1
`$match`	Filtrado de campos, similar a `where`	N:1
`$group`	Para agrupar los datos, similar a `group by`	N:1
`$sort`	Ordenar	1:1
`$skip`	Saltar	N:1
`$limit`	Limitar los resultados	N:1
`$unwind`	Separa los datos que hay dentro de un array	1:N

Preparando los ejemplos

Para los siguientes ejemplos, vamos a utilizar una colección de productos de una tienda de electrónica con las características y precios de los mismos.

```
[
{ nombre: 'iPad 16GB Wifi', fabricante: 'st', categoria: 'Tablets', precio: 499.00 },
```

```
{ nombre: 'iPad 32GB Wifi', fabricante: 'Apple', categoria: 'Tablets', precio: 599.00 },
{ nombre: 'iPad 64GB Wifi', fabricante: 'Apple', categoria: 'Tablets', precio: 699.00 },
{ nombre: 'Galaxy S3', fabricante: 'Samsung', categoria: 'Smartphones', precio: 563.99 },
{ nombre: 'Galaxy Tab 10', fabricante: 'Samsung', categoria: 'Tablets', precio: 450.99 },
{ nombre: 'Vaio', fabricante: 'Sony', categoria: 'Portátiles', precio: 499.00 },
{ nombre: 'Macbook Air 13inch', fabricante: 'Apple', categoria: 'Portátiles', precio: 499.00
},
{ nombre: 'Nexus 7', fabricante: 'Google', categoria: 'Tablets', precio: 199.00 },
{ nombre: 'Kindle Paper White', fabricante: 'Amazon', categoria: 'Tablets', precio: 129.00 },
{ nombre: 'Kindle Fire', fabricante: 'Amazon', categoria: 'Tablets', precio: 199.00 }
]
```

Un ejemplo de un producto sería:

```
> db.productos.findOne()
{
  "_id" : ObjectId("5345afc1176f38ea4eda4787"),
  "nombre" : "iPad 16GB Wifi",
  "fabricante" : "Apple",
  "categoria" : "Tablets",
  "precio" : 499
}
```

Para cargar este archivo desde la consola nos podemos conectar a nuestro clúster y copiar el contenido y pegarlo directamente en el *shell* de *MongoDB Compass* (son pocos datos).

$group

La fase group agrupa los documentos con el propósito de calcular valores agregados de una colección de documentos. Por ejemplo, podemos usar `$group` para calcular la media de páginas visitadas de manera diaria.

Cuidado

La salida de `$group` esta desordenada

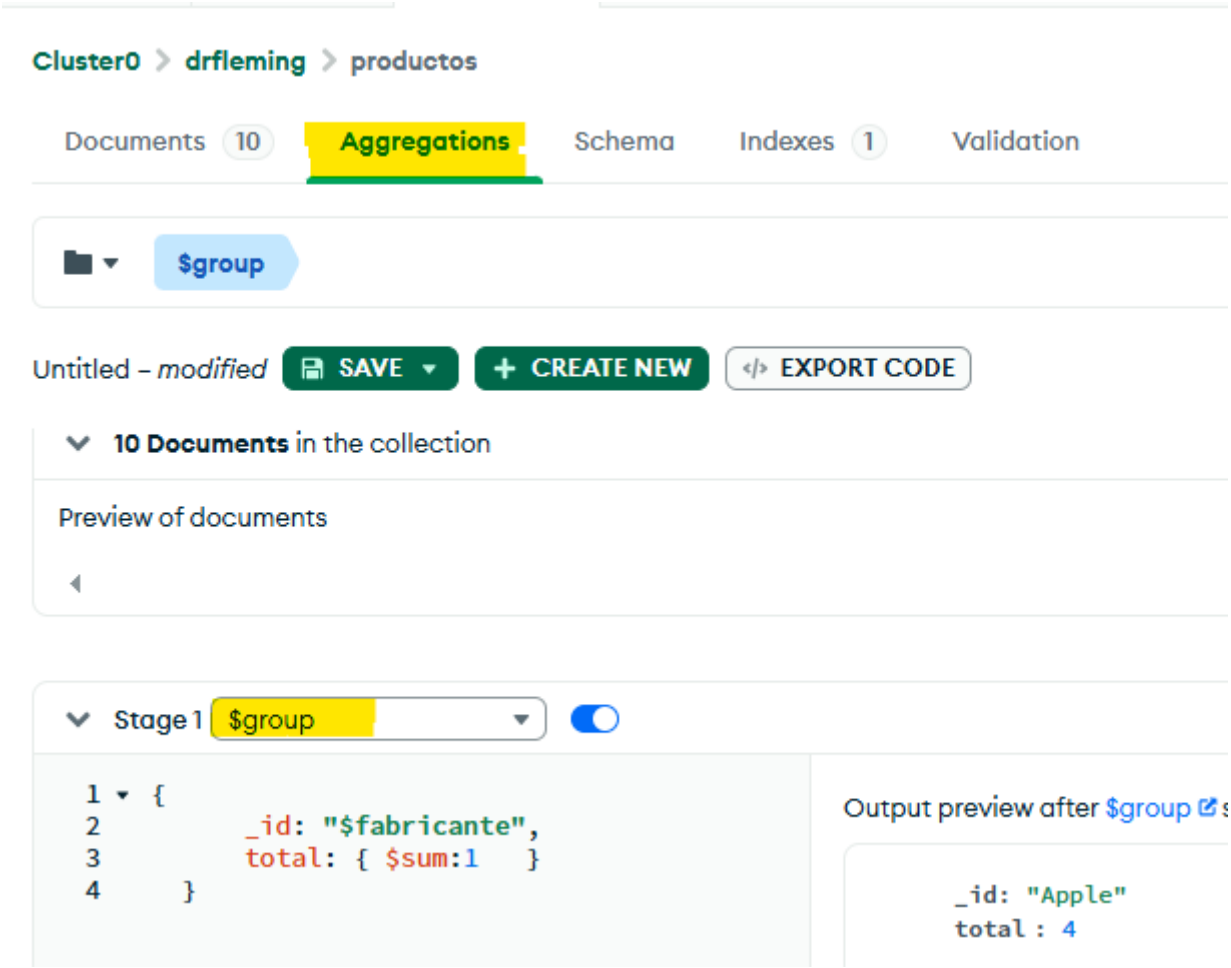
La salida de `$group` depende de cómo se definan los grupos. Se empieza especificando un identificador (por ejemplo, un campo `_id`) para el grupo que creamos con el *pipeline*. Para este campo `_id`, podemos especificar varias expresiones, incluyendo un único campo proveniente de un documento del *pipeline*, un valor calculado de una fase anterior, un documento con muchos campos y otras expresiones válidas, tales como constantes o campos de subdocumentos. También podemos usar operadores de `$project` para el campo `_id`.

Cuando referenciemos al valor de un campo lo haremos poniendo entre comillas un `$` delante del nombre del campo. Así pues, para referenciar al fabricante de un producto lo haremos mediante `$fabricante`.

```
> db.productos.aggregate([
  { $group: {
      _id: "$fabricante",
      total: { $sum:1 }
    }
  }])
< { _id: 'Apple', total: 4 }
  { _id: 'Samsung', total: 2 }
  { _id: 'Sony', total: 1 }
```

```
  { _id: 'Google', total: 1 }
  { _id: 'Amazon', total: 2 }
```

Para hacer agregaciones desde el entorno gráfico (Atlas o Compass) iremos al apartado de la imagen "*Aggregations*" crear un nuevo "*Stage*" seleccionando el operador "*Group*" en nuestro caso:



Si lo que queremos es que el valor del identificador contenga un objeto, lo podemos hacer asociándolo como valor:

```
> db.productos.aggregate([
  { $group: {
      _id: { "empresa": "$fabricante" },
      total: { $sum:1 }
    }
  }])
<
  { _id: { empresa: 'Sony' }, total: 1 }
  { _id: { empresa: 'Apple' }, total: 4 }
  { _id: { empresa: 'Google' }, total: 1 }
  { _id: { empresa: 'Samsung' }, total: 2 }
  { _id: { empresa: 'Amazon' }, total: 2 }
```

También podemos agrupar más de un atributo, de tal modo que tengamos un `_id` compuesto. Por ejemplo:

```
> db.productos.aggregate([
  { $group: {
```

```
      _id: {
        "empresa": "$fabricante",
        "tipo": "$categoria" },
      total: {$sum:1}
    }
  }])
< { _id: { empresa: 'Apple', tipo: 'Tablets' }, total: 3 }
  { _id: { empresa: 'Sony', tipo: 'Portátiles' }, total: 1 }
  { _id: { empresa: 'Apple', tipo: 'Portátiles' }, total: 1 }
  { _id: { empresa: 'Samsung', tipo: 'Smartphones' }, total: 1 }
  { _id: { empresa: 'Amazon', tipo: 'Tablets' }, total: 2 }
  { _id: { empresa: 'Google', tipo: 'Tablets' }, total: 1 }
  { _id: { empresa: 'Samsung', tipo: 'Tablets' }, total: 1 }
```

Siempre `_id`

Cada expresión de `$group` debe especificar un campo `_id`.

Acumuladores

Además del campo `_id`, la expresión `$group` puede incluir campos calculados. Estos otros campos deben utilizar uno de los siguientes acumuladores:

Nombre	Descripción
`$addToSet`	Devuelve un array con todos los valores únicos para los campos seleccionados entre cada documento del grupo (sin repeticiones)
`$first`	Devuelve el primer valor del grupo. Se suele usar después de ordenar.
`$last`	Devuelve el último valor del grupo. Se suele usar después de ordenar.
`$max`	Devuelve el mayor valor de un grupo
`$min`	Devuelve el menor valor de un grupo.
`$avg`	Devuelve el promedio de todos los valores de un grupo
`$push`	Devuelve un array con todos los valores del campo seleccionado entre cada documento del grupo (puede haber repeticiones)
`$sum`	Devuelve la suma de todos los valores del grupo

A continuación, vamos a ver ejemplos de algunos de estos acumuladores.

$SUM

El operador `$sum` acumula los valores y devuelve la suma.

Por ejemplo, para obtener el montante total de los productos agrupados por fabricante, haríamos:

```
> db.productos.aggregate([{
    $group: {
      _id: {
        "empresa": "$fabricante"
      },
      totalPrecio: {$sum:"$precio"}
    }
  }])
< { _id: { empresa: 'Apple' }, totalPrecio: 2296 }
  { _id: { empresa: 'Samsung' }, totalPrecio: 1014.98 }
  { _id: { empresa: 'Sony' }, totalPrecio: 499 }
  { _id: { empresa: 'Google' }, totalPrecio: 199 }
```

```
  { _id: { empresa: 'Amazon' }, totalPrecio: 328 }
```

$AVG

Mediante $avg podemos obtener el promedio de los valores de un campo numérico.

Por ejemplo, para obtener el precio medio de los productos agrupados por categoría, haríamos:

```
> db.productos.aggregate([{
    $group: {
      _id: {
        "categoria":"$categoria"
      },
      precioMedio: {$avg:"$precio"}
    }
  }])
< { _id: { categoria: 'Smartphones' }, precioMedio: 563.99 }
  { _id: { categoria: 'Portátiles' }, precioMedio: 499 }
  { _id: { categoria: 'Tablets' }, precioMedio: 396.4271428571428 }
```

$ADDTOSET

Mediante $addToSet obtendremos un array con todos los valores únicos para los campos seleccionados entre cada documento del grupo (sin repeticiones).

Por ejemplo, para obtener para cada empresa las categorías en las que tienen productos, haríamos:

```
> db.productos.aggregate([{
    $group: {
      _id: {
      "fabricante":"$fabricante"
      },
      categorias: {$addToSet:"$categoria"}
    }
  }])
< { _id: { fabricante: 'Apple' }, categorias: [ 'Portátiles', 'Tablets' ] }
  { _id: { fabricante: 'Amazon' }, categorias: [ 'Tablets' ] }
  { _id: { fabricante: 'Sony' }, categorias: [ 'Portátiles' ] }
  { _id: { fabricante: 'Google' }, categorias: [ 'Tablets' ] }
  { _id: { fabricante: 'Samsung' }, categorias: [ 'Tablets', 'Smartphones' ]
}
```

$PUSH

Mediante $push también obtendremos un array con todos los valores para los campos seleccionados entre cada documento del grupo, pero con repeticiones. Es decir, funciona de manera similar a `$addToSet` pero permitiendo elementos repetidos.

Por ello, si reescribimos la consulta anterior, pero haciendo uso de `$push`, obtendremos categorías repetidas:

```
> db.productos.aggregate([{
  $group: {
    _id: {
      "empresa":"$fabricante"
    },
    categorias: {$push:"$categoria"}
```

```
  }
}])
< { _id: { empresa: 'Sony' }, categorias: [ 'Portátiles' ] }
  { _id: { empresa: 'Apple' }, categorias: [ 'Tablets', 'Tablets', 'Tablets',
'Portátiles' ] }
  { _id: { empresa: 'Google' }, categorias: [ 'Tablets' ] }
  { _id: { empresa: 'Samsung' }, categorias: [ 'Smartphones', 'Tablets' ] }
  { _id: { empresa: 'Amazon' }, categorias: [ 'Tablets', 'Tablets' ] }
```

$MAX Y $MIN

Los operadores $max y $min permiten obtener el mayor y el menor valor, respectivamente, del campo por el que se agrupan los documentos.

Por ejemplo, para obtener el precio del producto más caro y más barato que tiene cada empresa haríamos:

```
> db.productos.aggregate([{
  $group: {
    _id: {
      "empresa":"$fabricante"
    },
    precioMaximo: {$max:"$precio"},
    precioMinimo: {$min:"$precio"},
  }
}])
{ "_id" : { "empresa" : "Amazon" }, "precioMaximo" : 199, "precioMinimo" :
129 }
{ "_id" : { "empresa" : "Sony" }, "precioMaximo" : 499, "precioMinimo" : 499
}
{ "_id" : { "empresa" : "Samsung" }, "precioMaximo" : 563.99, "precioMinimo"
: 450.99 }
{ "_id" : { "empresa" : "Google" }, "precioMaximo" : 199, "precioMinimo" :
199 }
{ "_id" : { "empresa" : "Apple" }, "precioMaximo" : 699, "precioMinimo" : 499
}
```

DOBLE $GROUP

Si queremos obtener el resultado de una agrupación podemos aplicar el operador `$group` sobre otro `$group`.

Por ejemplo, para obtener el precio medio de los precios medios de los tipos de producto por empresa haríamos:

```
> db.productos.aggregate([
    {$group: {
      _id: {
        "empresa":"$fabricante",
        "categoria":"$categoria"
        },
      precioMedio: {$avg:"$precio"}
      }
    },
    {$group: {
```

```
      _id: "$_id.empresa",
      precioMedio: {$avg: "$precioMedio"}
      }
    }
  ])
< { _id: 'Google', precioMedio: 199 }
  { _id: 'Sony', precioMedio: 499 }
  { _id: 'Samsung', precioMedio: 507.49 }
  { _id: 'Apple', precioMedio: 549 }
  { _id: 'Amazon', precioMedio: 164 }
```

$project

Si queremos realizar una proyección sobre el conjunto de resultados y quedarnos con un subconjunto de los campos usaremos el operador `$project`. Como resultado obtendremos el mismo número de documentos, y en el orden indicado en la proyección.

La proyección dentro del *framework* de agregación es mucho más potente que dentro de las consultas normales. Se emplea para:

- renombrar campos.
- introducir campos calculados en el documento resultante mediante `$add`, `$substract`, `$multiply`, `$divide` o `$mod`
- transformar campos a mayúsculas `$toUpper` o minúsculas `$toLower`, concatenar campos mediante `$concat` u obtener subcadenas con `$substr`.
- transformar campos en base a valores obtenidos a partir de una condición mediante expresiones lógicas con los operadores de comparación vistos en las consultas.

```
> db.productos.aggregate([
  {$project:
    {
      _id: 0,
      "empresa": { "$toUpper": "$fabricante" },
      "detalles": {
        "categoria": "$categoria",
        "precio": { "$multiply": ["$precio", 1.1] }
      },
      "elemento": "$nombre"
    }
  }
])
< { empresa: 'APPLE',
    detalles: { categoria: 'Tablets', precio: 548.9000000000001 },
    elemento: 'iPad 16GB Wifi' }
  { empresa: 'APPLE',
    detalles: { categoria: 'Tablets', precio: 658.9000000000001 },
    elemento: 'iPad 32GB Wifi' }
  ...
```

$match

El operador `$match` se utiliza principalmente para filtrar los documentos que pasarán a la siguiente etapa del *pipeline* o a la salida final.

Por ejemplo, para seleccionar sólo las *tablets* haríamos:

```
db.productos.aggregate([{$match:{categoria:"Tablets"}}])
```

Aparte de igualar un valor a un campo, podemos emplear los operadores usuales de consulta, como `$gt`, `$lt`, `$in`, etc…

Se recomienda poner el operador `$match` al principio del *pipeline* para limitar los documentos a procesar en siguientes fases. Si usamos este operador como primera fase podremos hacer uso de los índices de la colección de una manera eficiente.

Así pues, para obtener la cantidad de tablets de menos de 500 euros que tiene cada empresa haríamos:

```
> db.productos.aggregate([
    {$match:
      {categoria:"Tablets",
      precio: {$lt: 500}}},
    {$group:
      {_id: {"empresa":"$fabricante"},
      cantidad: {$sum:1}}
    }
  ])
< { _id: { empresa: 'Samsung' }, cantidad: 1 }
  { _id: { empresa: 'Amazon' }, cantidad: 2 }
  { _id: { empresa: 'Google' }, cantidad: 1 }
  { _id: { empresa: 'Apple' }, cantidad: 1 }
```

Autoevaluación

En la consulta anterior, si cambiamos el orden y hacemos el `$group` antes que el `$match`, ¿qué estamos obteniendo?

$sort

El operador `$sort` ordena los documentos recibidos por el campo, y el orden indicado por la expresión indicada al *pipeline*.

Por ejemplo, para ordenar los productos por precio descendentemente haríamos:

```
db.productos.aggregate({$sort:{precio:-1}})
```

El operador `$sort` ordena los datos en memoria, por lo que hay que tener cuidado con el tamaño de los datos. Por ello, se emplea en las últimas fases del *pipeline*, cuando el conjunto de resultados es el menor posible.

Si retomamos el ejemplo anterior, y ordenamos los datos por el precio total tenemos:

```
> db.productos.aggregate([
    {$match:{categoria:"Tablets"}},
    {$group:
      {_id: {"empresa":"$fabricante"},
      totalPrecio: {$sum:"$precio"}}
    },
    {$sort:{totalPrecio:-1}}
  ])
< { _id: { empresa: 'Apple' }, totalPrecio: 1797 }
  { _id: { empresa: 'Samsung' }, totalPrecio: 450.99 }
  { _id: { empresa: 'Amazon' }, totalPrecio: 328 }
  { _id: { empresa: 'Google' }, totalPrecio: 199 }
```

Un operador muy relacionado es $sortByCount. Este operador es similar a realizar las siguientes operaciones:

```
{ $group: { _id: <expresion>, cantidad: { $sum: 1 } } },
{ $sort: { cantidad: -1 } }
```

Así pues, podemos reescribir la consulta que hemos hecho en el operador `$group`:

```
db.productos.aggregate([
  { $group: {
      _id: "$fabricante",
      total: { $sum:1 }
    }
  },
  {$sort: {"total": -1}}
])
```

Y hacerla con:

Consulta

```
db.productos.aggregate([{ $sortByCount: "$fabricante"}])
```

Resultado

```
{ _id: 'Apple', count: 4 }
{ _id: 'Samsung', count: 2 }
{ _id: 'Amazon', count: 2 }
{ _id: 'Sony', count: 1 }
{ _id: 'Google', count: 1 }
```

$skip y $limit

El operador $limit únicamente limita el número de documentos que pasan a través del *pipeline*.

El operador recibe un número como parámetro:

```
db.productos.aggregate([{$limit:3}])
```

Este operador no modifica los documentos, sólo restringe quien pasa a la siguiente fase.

De manera similar, con el operador $skip, saltamos un número determinado de documentos:

```
db.productos.aggregate([{$skip:3}])
```

El orden en el que empleemos estos operadores importa y mucho, ya que no es lo mismo saltar y luego limitar, donde la cantidad de elementos la fija `$limit`:

```
> db.productos.aggregate([{$skip:2}, {$limit:3}])
< { _id: ObjectId("635194b32e6059646a8e7fee"),
    nombre: 'iPad 64GB Wifi',
    categoria: 'Tablets',
    fabricante: 'Apple',
    precio: 699 }
  { _id: ObjectId("635194b32e6059646a8e7fef"),
    nombre: 'Galaxy S3',
    categoria: 'Smartphones',
```

```
    fabricante: 'Samsung',
    precio: 563.99 }
  { _id: ObjectId("635194b32e6059646a8e7ff0"),
    nombre: 'Galaxy Tab 10',
    categoria: 'Tablets',
    fabricante: 'Samsung',
    precio: 450.99 }
```

En cambio, si primero limitamos y luego saltamos, la cantidad de elementos se obtiene de la diferencia entre el límite y el salto:

```
> db.productos.aggregate([{$limit:3}, {$skip:2}])
> { _id: ObjectId("635194b32e6059646a8e7fee"),
    nombre: 'iPad 64GB Wifi',
    categoria: 'Tablets',
    fabricante: 'Apple',
    precio: 699 }
```

$sample

Si tenemos un *dataset* muy grande, y queremos probar las consultas con un número reducido de documentos, podemos emplear el operador `$sample` y reducir la cantidad de documentos de manera aleatoria:

```
db.productos.aggregate([ { $sample: { size: 3 } } ])
```

$lookup

Si necesitamos unir los datos de dos colecciones, emplearemos el operador $lookup, el cual realiza un *left outer join* a una colección de la misma base de datos para filtrar los documentos de la colección *joineada*.

El resultado es un nuevo campo array para cada documento de entrada, el cual contiene los documentos que cumplen el criterio del *join*.

El operador `$lookup` utiliza cuatro parámetros:

- `from`: colección con la que se realiza el *join*.
- `localField`: campo de la colección origen, la que viene de la agregación `db.origen.aggregate`, que hace la función de clave ajena.
- `foreignField`: campo en la colección indicada en `from` que permite la unión (sería la clave primaria de la colección sobre la que se realiza el *join*).
- `as`: nombre del array que contendrá los documentos enlazados.

Preparando los datos

Vamos a utilizar la colección `zips` empleada en anteriores sesiones la cual tiene una estructura similar a:

```
{ _id: ObjectId("5c8eccc1caa187d17ca6ed18"),
  city: 'ACMAR',
  zip: '35004',
  loc: { y: 33.584132, x: 86.51557 },
  pop: 6055,
  state: 'AL' }
```

A continuación, vamos a crear una nueva colección llamada `state` con el nombre de los estados (states.js, ***- lo podéis encontrar en el material del curso -***), la cual cargaremos en la base de datos `sample_training`:

```
db.states.insertMany([
  {
      "name": "Alabama",
      "abbreviation": "AL"
```

```
  },
  {
      "name": "Alaska",
      "abbreviation": "AK"
  },
  ...
])
```

Vamos a analizar cómo funciona el operador `$lookup` mediante un ejemplo. Primero vamos a recuperar los tres estados más poblados. Para ello, podríamos hacer la siguiente consulta agregada:

Consulta

```
db.zips.aggregate([
  {$group: {
      _id: "$state",
      "totalPoblacion": {$sum:"$pop"}
  }},
  {$sort:{"totalPoblacion":-1}} ,
  {$limit: 3}
])
```

Resultado

```
{ _id: 'CA', totalPoblacion: 29760021 }
{ _id: 'NY', totalPoblacion: 17990455 }
{ _id: 'TX', totalPoblacion: 16986510 }
```

Si ahora queremos recuperar el nombre de esos tres estados, añadimos una nueva fase:

```
db.zips.aggregate([
  {$group: {
      _id: "$state",
      "totalPoblacion": {$sum:"$pop"}
  }},
  {$sort:{"totalPoblacion":-1}} ,
  {$limit: 3},
  {$lookup: {
    from: "states",
    localField: "_id",
    foreignField: "abbreviation",
    as: "estados"
  }},
])
```

Y ahora obtenemos para cada documento, un array con los documentos que coinciden (en este caso es una relación 1:1, y por eso cada array sólo contiene un elemento):

```
{ _id: 'CA',
  totalPoblacion: 29760021,
  estados:
   [ { _id: ObjectId("63565cd82889ecee358e0cd5"),
       name: 'California',
       abbreviation: 'CA' } ] }
{ _id: 'NY',
```

```
  totalPoblacion: 17990455,
  estados:
   [ { _id: ObjectId("63565cd82889ecee358e0cf4"),
       name: 'New York',
       abbreviation: 'NY' } ] }
{ _id: 'TX',
  totalPoblacion: 16986510,
  estados:
   [ { _id: ObjectId("63565cd82889ecee358e0d02"),
       name: 'Texas',
       abbreviation: 'TX' } ] }
```

Como la relación siempre va a provocar la creación de un array, mediante `$unwind`, lo podemos deshacer:

Consulta

```
db.zips.aggregate([
  {$group: {
      _id: "$state",
      "totalPoblacion": {$sum:"$pop"}
  }},
  {$sort:{"totalPoblacion":-1}} ,
  {$limit: 3},
  {$lookup: {
    from: "states",
    localField: "_id",
    foreignField: "abbreviation",
    as: "estados"
  }},
  {$unwind:"$estados"}
])
```

Resultado

```
{ _id: 'CA', totalPoblacion: 29760021, estados: { _id:
ObjectId("63565cd82889ecee358e0cd5"), name: 'California', abbreviation: 'CA'
} }

{ _id: 'NY', totalPoblacion: 17990455, estados: { _id:
ObjectId("63565cd82889ecee358e0cf4"), name: 'New York', abbreviation: 'NY' }
}

{ _id: 'TX', totalPoblacion: 16986510, estados: { _id:
ObjectId("63565cd82889ecee358e0d02"), name: 'Texas', abbreviation: 'TX' } }
```

Así pues, para finalmente obtener el nombre de cada estado, mediante `$project` recuperamos el campo `name`:

```
db.zips.aggregate([
  {$group: {
      _id: "$state",
      "totalPoblacion": {$sum:"$pop"}
  }},
  {$sort:{"totalPoblacion":-1}} ,
```

```
  {$limit: 3},
  {$lookup: {
    from: "states",
    localField: "_id",
    foreignField: "abbreviation",
    as: "estados"
  }},
  {$unwind:"$estados"},
  {$project: {
    "estado": "$estados.name",
    "poblacion": "$totalPoblacion"
  }}
])
```

Obteniendo el resultado deseado:

```
{ _id: 'CA', estado: 'California', poblacion: 29760021 }
{ _id: 'NY', estado: 'New York', poblacion: 17990455 }
{ _id: 'TX', estado: 'Texas', poblacion: 16986510 }
```

Persistiendo los resultados

Una vez hemos realizado nuestras consultas mediante el *framework* de agregación, es muy posible que queramos almacenar el resultado en una nueva colección para poder volver a consultar el resultado sin necesidad de ejecutar todas las fases.

Para ello, podemos emplear los operadores:

- `$out` recoge los documentos de una agregación y los persiste en una colección, sobrescribiendo los datos existentes.
- `$merge` similar a `$out`, pero permite añadir el resultado a la misma colección y además soporta trabajar con colecciones particionadas.

Por ejemplo, vamos a basarnos en las consultas con *join*, para crear una nueva colección con la población total de todos dos estados, y la vamos a almacenar en una nueva colección denominada `states_population`:

```
db.zips.aggregate([
  {$group: {
      _id: "$state",
      "totalPoblacion": {$sum:"$pop"}
  }},
  {$lookup: {
    from: "states",
    localField: "_id",
    foreignField: "abbreviation",
    as: "estados"
  }},
  {$unwind:"$estados"},
  {$project: {
    "estado": "$estados.name",
    "poblacion": "$totalPoblacion"
  }},
  {$out: "states_population"}
])
```

Tras su ejecución, podemos recuperar los datos:

```
> db.states_population.findOne()
< { _id: 'WY', estado: 'Wyoming', poblacion: 453588 }
```

De SQL al Pipeline de agregaciones

Ya hemos visto que el *pipeline* ofrece operadores para realizar la misma funcionalidad de agrupación que ofrece SQL.

Si relacionamos los comandos SQL con el *pipeline* de agregaciones tenemos las siguientes equivalencias:

SQL	**Pipeline de Agregaciones**
`WHERE`	`$match`
`GROUP BY`	`$group`
`HAVING`	`$match`
`SELECT`	`$project`
`ORDER BY`	`$sort`
`LIMIT`	`$limit`
`SUM()`	`$sum`
`COUNT()`	`$sum` / `$sortByCount`
join	`$lookup`

Podemos encontrar ejemplos de consultas SQL transformadas al *pipeline* en https://www.mongodb.com/docs/manual/reference/sql-aggregation-comparison/

Limitaciones

Hay que tener en cuenta las siguientes limitaciones:

- En versiones anteriores a la 2.6, el *pipeline* devolvía en cada fase un objeto BSON, y por tanto, el resultado estaba limitado a 16MB
- Actualmente, sólo cada documento que forme parte del resultado final debe ocupar menos de 16MB.
- Las fases tienen un límite de 100MB en memoria. Antes de la versión 6.0, si una consulta superaba el límite se producía un error, a no ser que se habilitara el uso de disco de manera explícita mediante `{allowDiskUse: true}` en cada consulta. Desde la versión 6.0, a partir de los 100MB se utilizan ficheros temporales en disco por defecto.

Más información en https://www.mongodb.com/docs/manual/core/aggregation-pipeline-limits/

Referencias

- Manual de MongoDB
- Cheatsheet oficial
- Comparación entre SQL y MongoDB
- Cursos gratuitos de Mongo University